Unit-тестирование

О себе

Алексей Солдатов

- Backend-разработчик.
- В hh работаю с середины 2022 года.
- В коммерческой разработке с 2013-го.
- На Java пишу с 2015-го.

Организационная часть

План лекции

- Основы ~20 минут.
- JUnit ~50 минут.
- Перерыв 10 минут.
- Mockito ~30 минут.
- Запуск и метрики ~10 минут.
- Q&A ~10 минут.

О наличии вопросов по ходу лекции буду периодически спрашивать.

Вопросы очень желательно задавать голосом.

Общаемся на «ты».

Содержание лекции

- Основы 5
- JUnit 17
  - Основные аннтотации 19
  - Проверка результатов 26
  - Параметризованные тесты 34
- Mockito 39
- Запуск unit-тестов 46
- Проверка unit-тестов 48
- Ссылки 51

Основы

Что это

Процесс, который позволяет проверить работоспособность отдельных частей (модулей) кода программы.

В объектно-ориентированном программировании (ООП), обычно, модуль — класс или отдельный метод.

Пирамида тестирования

Зачем

- Поиск ошибок на ранней стадии.
- Облегчение рефакторинга.
- Документация.

Когда писать unit-тесты

- До написания кода (TDD — Test Driven Development).
- После написания кода.
- Перед исправлением ошибок.
- Перед рефакторингом.
- В любое свободное время. ;-)

Когда запускать

- При внесении изменений в код.
- При обновлении библиотек.
- Перед тем как отдать код на ревью.
- В процессе CI/CD.

Основные принципы

- Изолированность.
- Единственная ответственность.
- Повторяемость.
- Независимость от порядка выполнения.
- Высокая скорость выполнения.
- Простота и понятность.

Что тестировать

В идеальном мире — все.

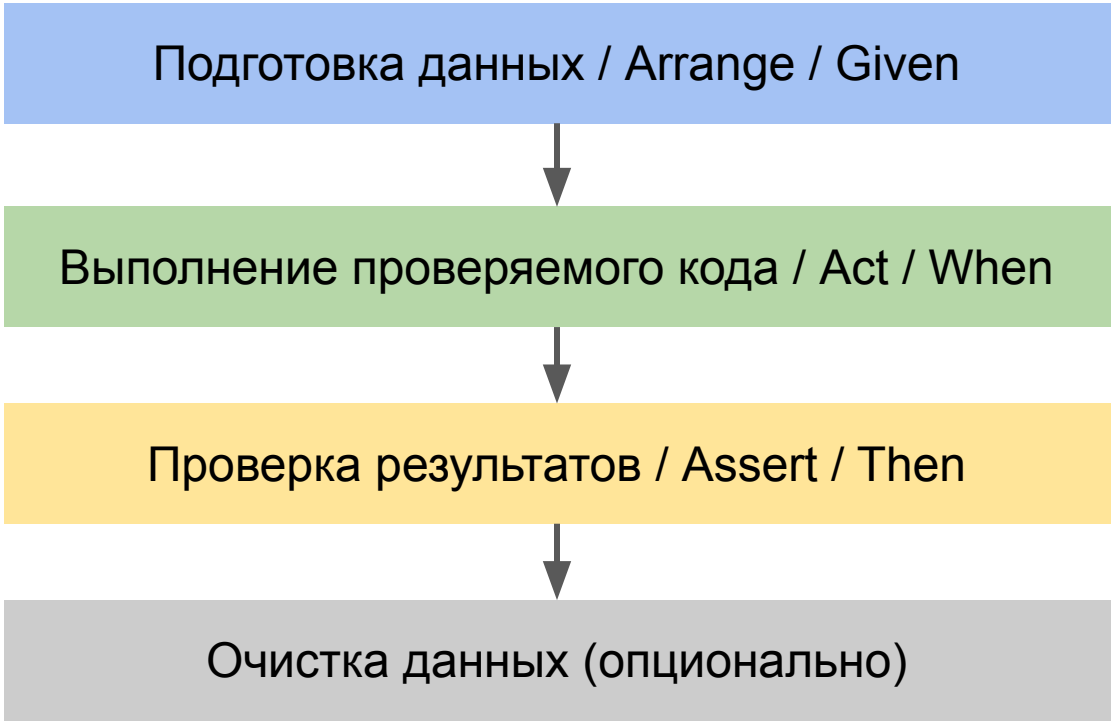
В реальном:

- Бизнес-логику.
- Любой сложный код.

Что можно не тестировать

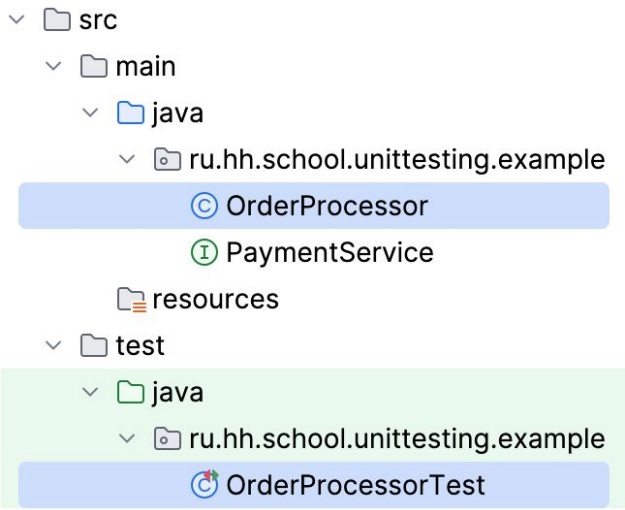
- Внешние библиотеки.
- Тривиальный код (например, геттеры и сеттеры).

Структура unit-теста



Организация и нейминг

- Обычно лежат в папке `/src/test/java`.
- Обычно в таких же пакетах, что и тестируемые классы.
- Чаще всего тестовый класс называется так же как и тестируемый класс, но с суффиксом `Test` (например, `SomeAwsomeServiceTest`).
- Название тестового метода может быть длинным, но должно отражать то, что в нем проверяется (например, `shouldReturnNullIfJsonIsIncorrect`).



Несколько популярных конвенций: https://dzone.com/articles/7-popular-unit-test-naming

Популярные фреймворки

- JUnit 4 — старая версия, но все еще можно встретить в проектах.
- JUnit 5 — улучшенная архитектура, новые аннотации, более широкие возможности.
- JUnit 6 — последняя на текущий момент версия JUnit. Изменения, в основном, внутренние. По большей части совместима с 5-й версией.
- TestNG — Имеет широкие возможности по конфигурированию и запуску тестов. Можно настроить параллельное выполнение. Удобно использовать для написания интеграционных или end-to-end тестов.

JUnit

Подключение к Maven-проекту

Добавляем в pom.xml зависимость:

```
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>6.0.3</version>
    <scope>test</scope>
</dependency>
```

Maven Repository: https://mvnrepository.com/artifact/org.junit.jupiter/junit-jupiter

Основные аннотации

Порядок вызова

@BeforeAll	Один раз перед всеми тестовыми методами в классе.
↓	
@BeforeEach	Перед каждым тестовым методом.
↓	
@Test	Тестовый метод.
↓	
@AfterEach	После каждого тестового метода.
↓	
@AfterAll	Один раз после всех тестовых методов в классе.

@BeforeAll

Метод с `@BeforeAll`:

- Вызывается **один раз перед всеми** тестами в классе.
- Используется для подготовки окружения.
- Метод должен быть статическим, но это можно обойти, если над классом повесить аннотацию `@TestInstance(TestInstance.Lifecycle.PER_CLASS)`.

```
@BeforeAll
static void beforeAll() {
  setupEnvironment();
}
```

@BeforeEach

Метод с `@BeforeEach`:

- Вызывается **перед каждым** тестовым методом.
- Используется для подготовки тестовых данных, которые пригодятся во всех тестах.

```
@BeforeEach
void setUp() {
  createCommonTestData();
}
```

@Test

- Лежит в пакете `org.junit.jupiter.api`.
- Помечает метод как тестовый.
- При прогоне тестов будут запущены все методы с этой аннотацией.
- Можно временно отключить аннотацией `@Disabled`.

```
@Test
void testSum() {
  var calculator = new Calculator();
  var result = calculator.sum(2, 3);
  assertEquals(5, result);
}
```

@AfterEach

Метод с `@AfterEach`:

- Вызывается **после каждого** тестового метода.
- Используется для очистки ресурсов.

```
@AfterEach
void tearDown() {
  cleanResources();
}
```

@AfterAll

Метод с `@AfterAll`:

- Вызывается **один раз после всех** тестов в классе.
- Используется для очистки ресурсов.
- Метод должен быть статическим, но это можно обойти, если над классом повесить аннотацию `@TestInstance(TestInstance.Lifecycle.PER_CLASS)`.

```
@AfterAll
static void afterAll() {
  cleanResources();
}
```

Проверка результатов

JUnit Assertions

- Методы для проверки результатов выполнения тестируемого кода.
- Лежат в классе `org.junit.jupiter.api.Assertions`.
- Начинаются со слова *`assert`*.
- В одном тесте может быть больше одной проверки.
- Тест считается проваленным, если не прошла хотя бы одна проверка.
- Если какая-то проверка не прошла, то тестовый метод дальше не выполняется.
- Предыдущий пункт можно обойти, завернув несколько проверок в метод *`assertAll`*.

assertEquals и asssertNotEquals

Сравнивает переданные аргументы между собой по equals.

```
@Test
void testSum() {
  var calculator = new Calculator();
  var result = calculator.sum(2, 3);
  assertEquals(5, result);
}

@Test
void shouldNotRepeatPasswords() {
  var passwordGenerator = new PasswordGenerator();
  var password1 = passwordGenerator.generate();
  var password2 = passwordGenerator.generate();
  assertNotEquals(password1, password2);
}
```

assertTrue и assertFalse

Проверяют, что переданный аргумент `true` или `false`.

```
@Test
void isEmptyShouldReturnTrue() {
  boolean isEmpty = StringUtils.isEmpty("");
  assertTrue(isEmpty);
}

@Test
void isEmptyShouldReturnFalse() {
  boolean isEmpty = StringUtils.isEmpty("non-empty string");
  assertFalse(isEmpty);
}
```

assertNull и assertNotNull

Проверяют, переданный аргумент на `null`.

```
@Test
void upperCaseShouldReturnNull() {
  var result = StringUtils.upperCase(null);
  assertNull(result);
}

@Test
void upperCaseShouldNotReturnNull() {
  var result = StringUtils.upperCase("lower case string");
  assertNotNull(result);
}
```

assertThrows и assertDoesNotThrow

Проверяют, кидает ли код исключение.

```
@Test
void shouldThrowExceptionOnEmptyParameters() {
  var parameters = new Parameters();
  var validator = new ParametersValidator();
  assertThrows(IllegalArgumentException.class, () -> validator.validate(parameters));
}

@Test
void shouldNotThrowExceptionOnCorrectParameters() {
  var parameters = new Parameters(1, 2, 3);
  var validator = new ParametersValidator();
  assertDoesNotThrow(() -> validator.validate(parameters));
}
```

assertAll

Выполняет все ассерты, которые в него переданы, вне зависимости от их успешности.

```
@Test
void testDefaultPersonName() {
  var person = Person.createDefault();
  assertAll(
    () -> assertEquals("John", person.getFirstName()),
    () -> assertEquals("Doe", person.getLastName())
  );
}
```

AssertJ

Библиотека с более гибкими и читаемыми ассертами с более широкими возможностями, чем стандартные в JUnit.

Например:

```
assertThat(calculator.sum(2, 3)).isEqualTo(5);
assertThat(person.getName()).startsWith("Jo").endsWith("hn");
assertThat(StringUtils.isEmpty("")).isTrue();
assertThatThrownBy(() -> validator.validate(parameters)).hasMessage("Invalid parameters");
assertThat(oneObject).usingRecursiveComparison().isEqualTo(anotherObjectWithSameNamedFields);
assertThat(list).containsAll(item1, item2, item3);
```

И многое другое: https://assertj.github.io/doc/

Параметризованные тесты

Основы

- Тестовые методы, которые запускаются несколько раз с разными параметрами.
- Используется аннотация `@ParametrizedTest` вместо `@Test`.
- У таких тестовых методов должны быть параметры и заданы источники данных через аннотации.

```
@ParametrizedTest
@ValueSource(strings = { "racecar", "radar", "able was I ere I saw elba" })
void testCandidateIsPalindrome(String candidate) {
  assertTrue(StringUtils.isPalindrome(candidate));
}
```

Источники единственного параметра

- `@ValueSource` — простейший источник для единственного параметра. Позволяет задавать массив значений примитивных типов, а также String и Class.
- `@NullSource`, `@EmptySource`, `@NullAndEmptySource` — источники пустых параметров.

```
@ParametrizedTest
@NullAndEmptySource
@ValueSource(strings = { " ", "  ", "\t", "\n" })
void parserShouldReturnNullOnBlankValue(String value) {
  var parser = new JsonParser();
  assertNull(parser.parse(value));
}
```

@CsvSource

- Позволяет задать параметры в виде строк в формате CSV.
- Можно указывать больше одного параметра.

```
@ParametrizedTest
@CsvSource({
    "2, 3, 5",
    "-2, -3, -5",
    "0, 0, 0",
})
void testSum(int a, int b, int expectedSum) {
  var calculator = new Calculator();
  var sum = calculator.sum(a, b);
  assertEquals(expectedSum, sum);
}
```

@MethodSource

- Позволяет указывать метод, в котором формируются параметры.
- Можно указывать больше одного параметра.
- Удобно использовать для сложных наборов параметров.

```
@ParametrizedTest
@MethodSource("personSource")
void testPersonJsonParser(String json, Person expected) {
  assertEquals(expected, PersonJsonParser.parse(json));
}

static Stream<Arguments> personSource() {
  return Stream.of(
      arguments("{\"firstName\":\"John\",\"lastName\":\"Doe\"}", new Person("John", "Doe")),
      arguments("{}", new Person(null, null)),
  );
}
```

Mockito

Что это

- Фреймворк для создания моков — имитаций реальных объектов.
- Позволяет изолировать тестовый код.
- Обычно используется для исключения внешних зависимостей из тестирования.

Подключение к Maven-проекту

Добавляем в pom.xml зависимость:

```
<dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-junit-jupiter</artifactId>
    <version>5.21.0</version>
    <scope>test</scope>
</dependency>
```

Maven Repository: https://mvnrepository.com/artifact/org.mockito/mockito-junit-jupiter

Аннотации

- `@Mock` — создает мок.
- `@InjectMocks` — создает экземпляр тестируемого класса и внедряет в него моки.
- `@Spy` — создает частичный мок.
- `@Captor` — создает перехватчик аргументов.

Чтобы аннотации работали:

- повесить `@ExtendWith(MockitoExtension.class)` над классом;
- либо выполнить `MockitoAnnotations.openMocks(this)` в `@BeforeEach`.

Создание мока

Аннотация @Mock:

```
import org.mockito.Mock;
// ...
@Mock
private UserService userService;
```

Метод Mockito.*mock*:

```
import static org.mockito.Mockito.mock;
// ...
private UserService userService;
// ...
userService = mock(UserService.class);
```

Имитация поведения

Задать поведение определенному методу:

```
Mockito.when(userService.findById(AgrumentMatchers.any()))
    .thenReturn(new User(1, "John"));
```

Тогда вызов в тесте `userService.findById` с любым параметром вернет `new User(1, "John")`.

У *`any()`* есть типизированные версии (например, *`anyInt()`*, *`any(type)`*), на случай если есть перегруженные методы с параметрами разных типов.

Вместо *`any()`* можно указывать реальные значения, но нельзя смешивать.

Конкретные значения можно указать через *`eq()`*.

Пример

```java
public record User (int id, boolean active) {}

public class UserRepository {
  public User findById(int id) {
    // ...
  }
}

public class UserService {
  private final UserRepository userRepository;

  public UserService(UserRepository userRepository) {
    this.userRepository = userRepository;
  }

  public boolean isUserActive(int id) {
    User user = userRepository.findById(id);
    return user != null && user.active();
  }
}
```

```java
@ExtendWith(MockitoExtension.class)
public class UserServiceTest {
  @Mock
  private UserRepository userRepository;
  @InjectMocks
  private UserService userService;

  @Test
  void userShouldBeActive() {
    when(userRepository.findById(1))
        .thenReturn(new User(1, true));
    assertTrue(userService.isUserActive(1));
  }
}
```

Запуск unit-тестов

Способы

- Вручную в IDE.

```
9   @Test
10  void testProcessOrderWithSufficientStock() {
```

- С помощью Maven с плагином maven-surefire-plugin: `mvn test`.

```
<plugin>
   <artifactId>maven-surefire-plugin</artifactId>
   <version>3.3.1</version>
</plugin>
```

- В процессе CI/CD.

Проверка unit-тестов

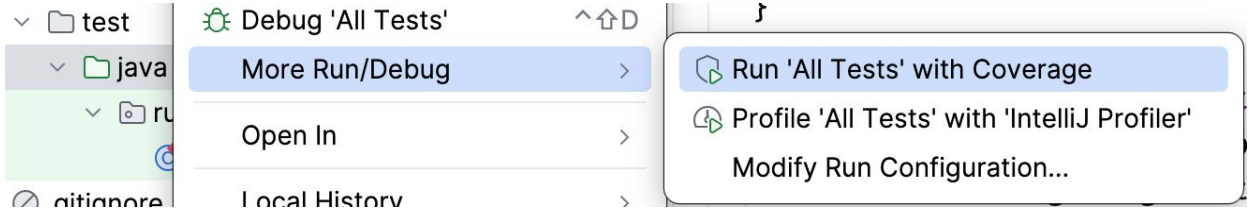
Покрытие кода

- Метрика, которая показывает, какое количество кода выполняется при запуске тестов.
- Обычно измеряется в процентах от общего количества строк, методов, классов или веток кода.

Element	Class, %	Method, %	Line, %	Branch, %
ru.hh.zp.vacancy.improver	81% (31/38)	73% (130/178)	75% (467/620)	76% (113/148)
config	33% (2/6)	50% (6/12)	35% (10/28)	50% (1/2)
dao	100% (2/2)	66% (12/18)	84% (134/158)	82% (53/64)
dto	100% (15/15)	97% (38/39)	95% (45/47)	62% (5/8)
kafka	66% (4/6)	24% (8/33)	39% (43/109)	20% (4/20)
model	50% (1/2)	90% (27/30)	90% (40/44)	100% (2/2)
resource	100% (1/1)	33% (1/3)	50% (3/6)	100% (0/0)
service	100% (5/5)	90% (37/41)	84% (188/223)	92% (48/52)
mapper	100% (3/3)	85% (12/14)	71% (60/84)	100% (8/8)
VacancyCrosspostingTasksService	100% (1/1)	94% (18/19)	92% (90/97)	89% (34/38)
VacancyCrosspostsStatisticsService	100% (1/1)	87% (7/8)	90% (38/42)	100% (6/6)
App	100% (1/1)	50% (1/2)	80% (4/5)	100% (0/0)

Примеры инструментов для измерения покрытия

- Встроенная функция в Intellij IDEA



- Maven-плагин Jacoco
  https://mvnrepository.com/artifact/org.jacoco/jacoco-maven-plugin
- Maven-плагин Pitest
  https://pitest.org/

Ссылки

- Документация JUnit: https://docs.junit.org
- JUnit Series на Baeldung: https://www.baeldung.com/junit
- Документация AssertJ: https://assertj.github.io/doc/
- Документация Mockito: https://javadoc.io/doc/org.mockito/mockito-core/latest/org/mockito/Mockito.html
- Mockito Series на Baeldung: https://www.baeldung.com/mockito-series
- 7 Popular Unit Test Naming Conventions: https://dzone.com/articles/7-popular-unit-test-naming
- Примеры из лекции: https://github.com/blackoutks/hh-school-2026-unit-testing

Спасибо за внимание!